



Skill Stuff · [Подписаться на публикацию](#)

Github +

Git +

Git Command Line +

Разработка программного обеспечения +

10 команд Git, которые помогли мне стать более быстрым разработчиком

4 мин на чтение · 6 дней назад



Шаян Хан

Подписаться



Слушать



Поделиться

10 GIT COMMANDS

— THAT MADE ME A —

FASTER DEVELOPER

Work smarter with Git.
Save time. Avoid mistakes. Ship faster.

```
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   index.js
```

FASTER WORKFLOW

FEWER MISTAKES

BETTER COLLABORATION

MORE PRODUCTIVE

Pro Tip: Master these 10 commands and you'll handle almost any Git workflow with confidence!

Happy Coding! 🚀

01		git status Check the status of your working directory and staging area.
02		git add . Stage all changes in the current directory.
03		git commit -m "message" Commit your staged changes with a message.
04		git pull Fetch and merge changes from the remote repository.
05		git push Push your committed changes to the remote repo.
06		git checkout -b branch-name Create a new branch and switch to it.
07		git merge branch-name Merge the specified branch into the current branch.
08		git log --oneline --graph --all View a clean, visual commit history.
09		git reset --hard HEAD Reset everything to the last commit. (Use carefully!)
10		git clean -fd Remove untracked files and directories.

Когда я только начал работать с Git, я знал всего несколько команд:

- git add

- `git commit`

Вы вышли из аккаунта. Войдите в свой аккаунт

- `git pull` ([sv__@g__.com](#)), чтобы просматривать истории, доступные только участникам. Вход

Этого было достаточно, но я часто искал информацию в интернете, когда мне нужно было исправить ошибку, переключиться между ветками или проверить изменения.

По мере работы над новыми проектами я постепенно освоил еще несколько команд, которые полностью изменили мой рабочий процесс. Сегодня я использую эти команды почти каждый день, и они помогают мне работать с Git гораздо быстрее и увереннее.

Вот 10 команд Git, которые больше всего повлияли на мою продуктивность.

1. `git status`

```
git status
```

Эту команду я использую чаще других.

Она показывает:

- Какие файлы изменились
- Какие файлы подготовлены к коммиту
- В какой ветке я нахожусь
- Опережает ли моя ветка другую или отстает от нее

Прежде чем что-то зафиксировать, я всегда проверяю `git status`. Это помогает избежать случайных коммитов и позволяет мне быть в курсе того, что происходит в моем репозитории.

2. git log

Вы вышли из аккаунта. Войдите в свой аккаунт (sv__@g__.com), чтобы просматривать истории, доступные только участникам. Вход

```
git log --oneline
```

Вместо подробного описания коммитов эта версия показывает чистый список коммитов.

Пример:

```
9e2b741 Добавить аутентификацию
f5c0d92 Исправить ошибку в навигационной панели
3a6d111 Начальная настройка проекта
```

Когда пытаешься вспомнить, что изменилось, гораздо проще просмотреть недавние изменения.

3. git diff

```
git diff
```

Перед коммитом я всегда проверяю различия.

Он показывает все строки, которые были добавлены, удалены или изменены.

Эта простая привычка спасала меня от коммита незавершенного кода столько раз, что и не сосчитать.

4. git checkout

Вы вышли из аккаунта. Войдите в свой аккаунт (sv__@g__.com), чтобы просматривать истории, доступные только участникам. Вход

```
git checkout feature/login
```

Although many developers now use `git switch`, I still encounter `git checkout` in existing projects.

It's useful for:

- Switching branches
- Restoring files
- Reviewing older commits

Knowing how it works is still valuable when collaborating on different codebases.

5. git switch

```
git switch feature/dashboard
```

This newer command has one purpose: switching branches.

I prefer it because it's more descriptive than `checkout`.

Get Shayan Khan's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

☐ Remember me for faster sign in

Вы вышли из аккаунта. Войдите в свой аккаунт
(sv__@g__.com), чтобы просматривать истории, доступные
только участникам. Вход

Creating a

```
git switch -c feature/profile
```

6. git stash

```
git stash
```

This command is a lifesaver when you're halfway through a feature and suddenly need to fix something urgent.

Instead of committing incomplete work, I temporarily save it:

```
git stash
```

Later, I restore it with:

```
git stash pop
```

It's one of those commands you don't appreciate until you need it.

7. git pull

Вы вышли из аккаунта. Войдите в свой аккаунт (sv__@g__.com), чтобы просматривать истории, доступные только участникам. Вход

```
git pull
```

Before starting work each day, I always pull the latest changes.

This keeps my local branch up to date and reduces the chances of running into merge conflicts later.

It's a simple habit that saves a lot of headaches.

8. git branch

```
git branch
```

This command lists every branch in the repository.

When projects become larger, it's an easy way to see where you're working and clean up branches that are no longer needed.

I also use it to create branches:

```
git branch feature/settings
```

9. git restore

```
git re:  Вы вышли из аккаунта. Войдите в свой аккаунт  
(sv__@g__.com), чтобы просматривать истории, доступные  
только участникам. Вход
```

Made changes you didn't mean to keep?

This command restores a file back to its previous state.

It's much safer than trying to manually undo every edit.

Just make sure the changes aren't something you wanted to save first.

10. `git cherry-pick`

```
git cherry-pick <commit-id>
```

Sometimes you only want one specific commit from another branch.

Instead of merging everything, `git cherry-pick` lets you copy just that single commit.

It's especially useful when fixing production bugs or moving small features between branches.

Bonus Tips

A few small habits have helped me avoid countless Git problems:

- Commit small, focused changes instead of huge batches.
- Write commit messages that clearly describe what changed.
- Create feature branches instead of working directly on the main branch.

- Pull the latest changes before starting new work.
- Review Вы вышли из аккаунта. Войдите в свой аккаунт (sv__@g__.com), чтобы просматривать истории, доступные только участникам. Вход

These habits are just as important as knowing the commands themselves.

Final Thoughts

Git can seem overwhelming when you're getting started, but you don't need to memorize dozens of commands to become productive.

Learning a small set of commands — and understanding when to use them — is enough to handle most day-to-day development tasks with confidence.

Even now, these are the commands I rely on the most. They help me work faster, recover from mistakes more easily, and collaborate with other developers without unnecessary stress.

If you're still only using `git add`, `git commit`, and `git push`, try adding a few of these commands to your daily workflow. You'll likely wonder how you managed without them.

Github

Git

Git Command Line

Software Development



Follow

Published in Skill Stuff

749 followers · Last published 2 hours ago

Skill Stuff is your go-to hub for coding tips, problem-solving strategies, and developer growth insights.



Вы вышли из аккаунта. Войдите в свой аккаунт (sv__@g__.com), чтобы просматривать истории, доступные только участникам. Вход

Follow

Written L, -----

31 followers · 20 following

UI/UX Designer & Web App Developer focused on designing clean, scalable enterprise dashboards and ERP systems

Вы вышли из аккаунта. Войдите в свой аккаунт
(**sv__@g__.com**), чтобы просматривать истории, доступные
только участникам. Вход

Вы вышли из аккаунта. Войдите в свой аккаунт
(**sv__@g__.com**), чтобы просматривать истории, доступные
только участникам. Вход